Lab 3 — Architecture Selection & Justification

Problem Statement #56: Peer Skill Exchange & Mentorship Network | Anchita Chhibba | SRN: PES2UG24CS060

Architecture Selection

We chose **Layered Architecture** for the **Peer Skill Exchange & Mentorship Network** System.

Architectural Choice

The system is organised into three horizontal layers: a **Presentation Layer** (User Interface Component), a **Business Layer** (Authentication & Access Control, Mentor Search, Session Manager, Time Credit Manager, Feedback & Rating) and a **Data & Integration Layer** (Database and Calendar Sync components, with Google Calendar / Outlook as the external system). Each use case from Lab 1 (UC-01 to UC-06) is handled by a single dedicated component, and components communicate only through their provided and required interfaces. **Alternatives considered:** Microservices would give independent scaling, but its network latency, operational complexity and data-consistency challenges are not justified for a community-scale platform like this one; Client-Server would be simple, but a single centralised server with no internal separation makes the rules hard to maintain and is a single point of failure.

Two Reasons for This Choice

- **1. Business rules must be enforced in one controlled place.** The time-banking flow (search → schedule → confirm → credit transfer → feedback) depends on strict rules: a credit moves only after both members confirm completion (FR-001, FR-004), a session is created only in a mutually free slot (FR-003), and feedback is allowed only after verification (FR-005). In a layered design these rules live in the Business Layer, so the UI cannot bypass them and the database cannot be updated out of order. The credit transfer also needs strong, immediate data consistency, which a layered, single-database design provides easily, whereas microservices make it difficult.
- **2. External calendar integration can change without affecting core logic.** NFR-001 requires iCal feeds for Google Calendar and Outlook. The Calendar Sync component hides this behind the ICalendarSync and IiCalFeed interfaces, so a change in calendar formats or providers is confined to that one component, and Session Manager is untouched. Because each component is a black box defined only by its interfaces, components can be replaced or tested in isolation, which suits a small development team.

Security Advantage

Every request must pass through the Authentication & Access Control component before reaching the business logic, and the Database is reachable only through the IDataAccess interface used by the Business Layer, never directly from the UI. This gives one central place to verify the member's token and to enforce that a member can read or modify only their own schedule and time-credit data (NFR-002), and one place to audit. Database queries are also parameterised in the data layer, reducing injection risk.

Performance Benefit

Business components are deployed together, so calls such as ICreditTransfer and ISessionStatus are in-process method calls rather than network hops, which keeps scheduling and confirmation responses fast (unlike microservices, which add network latency per call). Mentor search can also be optimised in one place, using an index on the skill field and caching popular searches, and the whole application can be scaled horizontally behind a load balancer if membership grows.